

2026 · 犀牛鸟开源人才培养

腾讯开源 · 开源课题实战

项目申请书

tRPC · 多租户与节点部署架构设计

牛晨勋

中南大学 · 数据科学与大数据技术 · 2023 级

 GITHUB  STELQUIS  CNB  STELQUIS  邮箱 17837343195@163.COM

"Always a learner, always learning from the best."

项目申请书

项目: tRPC · 多租户与节点部署 申请人: 牛晨勋 学校: 中南大学 · 数据科学与大数据技术 · 2023 级
邮箱: 17837343195@163.com / 3420761503@qq.com

一、背景与动机

tRPC-Agent-Python 当前提供了一套完整的 Agent 构建、编排、工具接入、会话记忆等能力，但在生产级部署上仍处于“单机可跑”的阶段。当需要将同一个 Agent 服务同时开放给多个业务方时，就暴露出几个现实问题：

- 不同租户的模型配置（一个用 DeepSeek、一个用 GPT-4）如何隔离？
- 每个租户的会话数据、知识库、审计日志是混在一起还是分开存？
- 如果 Agent 流量大了，能否横向加机器而不中断现有会话？

这些问题的本质是 多租户隔离 + 节点水平扩展 两个架构维度的缺失。我选择这个课题，是因为它恰好位于“框架功能完备”和“生产可用”之间的过渡地带——解决了它，Agent 框架才能真正从示例走向企业级服务。

前置学习 源码已读

实验 已跑通

PR 已提交

框架 tRPC-Agent-Python

二、技术方案

2.1 核心模块设计

整个方案围绕 四个核心模块 展开，每个模块解决一个独立的架构问题：

模块一：租户模型与配置隔离

设计 Tenant 数据模型，核心字段包括：`tenant_id`（唯一标识）、`model_config`（模型提供商、API Key 引用、模型名称）、`tool_whitelist`（允许使用的工具列表）、`channel_binding`（IM 通道配置）、`audit_policy`（审计等级与日志保留策略）。每个租户独立存储配置，运行时不交叉读取。

✂ 难点：API Key 不能明文存储，但又要能在运行时被 Worker 读取。方案是在 Tenant 表中只存 Key 的引用 ID，实际密钥由密钥管理服务持有，Worker 按需获取。

模块二：节点部署拓扑与无状态 Worker

设计四类节点：`Agent Gateway`（统一入口，按 `tenant_id + session_id` 路由请求）、`Agent Worker`（无状态，只负责执行 Agent 逻辑，不持有任何会话数据）、`Storage Adapter`（统一数据访问层）、`Telemetry Collector`（全链路追踪）。关键决策是：不依赖 `Sticky Session`——所有会话状态存入共享 Redis/SQL 后端，Worker 自由扩缩容。

突破口：tRPC-Agent 的 `SessionService` 抽象层本就是多后端可切换的，已有 `InMemory`、`SQL`、`Redis` 三种实现。`Storage Adapter` 只需要在此基础上包一层租户路由逻辑。

模块三：统一数据访问抽象

设计 `StorageRouter`，根据 `tenant_id` 查找该租户的后端配置，将请求路由到正确的存储实例。覆盖的数据类型：`Session` `Memory` `Summary` `Knowledge` `AuditLog`。

✂ 难点：不同后端的写入一致性模型不同——Redis 是最终一致，SQL 是强一致。需明确每个数据类型的一致性取舍。

模块四：IM Channel Adapter

设计 Adapter 接口，将外部 IM 消息统一转换为 `tRPC-Agent` 的 `NewMessage` 输入。计划覆盖企业微信（WeCom）和 Telegram 两类通道，重点处理消息验签、消息去重、用户身份映射、群聊/单聊的 `session_id` 生成规则。

2.2 可行性分析

乐观锁

并发写入一致性
Redis `WATCH/MULTI/EXEC`

双写切换

数据迁移方案
新旧后端同时写入 → 切换读流量

SETNX 去重

IM 消息幂等
5 分钟 TTL 去重缓存

三、时间规划

实战周期：2026 年 8 月 1 日 ~ 9 月 10 日

第 1 周

基础调研与设计

精读 A2A/AG-UI 服务端源码，设计 Tenant 数据模型，完成存储层抽象接口
→ Tenant 模型设计文档 + 抽象接口代码

第 2 周

⚙ 核心开发

实现 `StorageRouter` + Redis/SQL 双后端适配，实现 `Agent Gateway` 租户路由
→ 多租户存储层可运行 + 单元测试

第 3 周

IM 集成

实现企业微信 + Telegram Channel Adapter，完成消息转换与验签
→ 双通道 IM 适配器可运行

第 4 周

打磨交付

Filter 扩展（租户级工具白名单、预算限制）、Docker Compose 部署方案、PR 提交
→ 完整示例 + 部署文档 + PR

期望获得的成长与收益

架构设计能力

通过多租户与节点部署的实战，理解生产级 Agent 平台的架构设计范式——从“单机可跑”到“企业级服务”的演进路径。

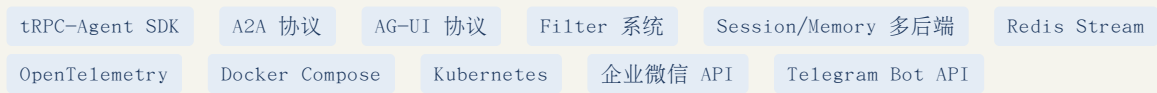
开源协作经验

深度参与 `tRPC` 社区的 PR 评审流程，学习与社区维护者的协作方式，理解开源项目的治理和贡献规范。

系统化思维

在一致性取舍、数据迁移、租户隔离等现实约束下做工程决策，提升从“功能实现”到“系统设计”的思维层次。

期望熟悉的技术与模块



四、开源经历

4.1 本次参与的 Issue 与心得体会

本次申请的是 tRPC 项目下的 多租户与节点部署 课题。在确定申请后，我做了大量的前置准备工作：



心得体会：

从源码阅读到动手实验，再到提交功能 PR，整个过程让我对 tRPC-Agent 框架有了从“用户”到“理解者”的转变。最有感触的是两点：

第一，理解框架的设计哲学比调用 API 更重要。读 `_llm_agent.py` 时我发现，Agent 的核心循环不是复杂的黑魔法，而是“LLM 推理 → 工具调用 → 结果回填 → 再次推理”的交替循环。理解了这一点，就能预判哪些地方可以扩展（Filter 链）、哪些地方需要谨慎修改（Tool Loop 的并发安全）。

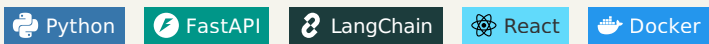
第二，开源贡献的价值不仅在“合入”。我提交的 3 个 PR 虽然尚未合并，但评审过程中维护者的反馈让我学到了很多：代码规范、测试覆盖、文档质量——这些是课堂上学不到的工程实践。

4.2 开发经验与技术栈

虽然没有长期参与其他开源项目，但通过以下项目的完整交付，我具备了扎实的上手能力：

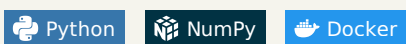
ChinaVis 2026 队长 2026.03 ~ 2026.06

- 搭建 30+ Python 脚本数据处理管线，处理 1,473 部非结构化剧本，提取 29 维角色特征、8 项网络拓扑指标
- 运用卡方检验、ANOVA、KMeans 聚类等 15+ 种统计方法，全部核心发现 $p < 0.001$
- FastAPI + LangChain 集成 DeepSeek 大模型，总代码量约 45,000 行



ACM 挑战赛 2025.12 ~ 2026.01

- 设计自适应预加载算法，综合得分提升 45.41%，带宽浪费降低 21.03%
- 核心算法仅 204 行，Docker + CNB 云原生环境，双平台自动化测试



4.3 技术栈掌握程度

技术领域	具体技能	掌握程度
Python 生态	FastAPI、异步编程、类型注解、pytest	精通
AI Agent 框架	tRPC-Agent SDK、LangChain、Tool Calling、Filter 机制	熟练
数据处理	NumPy、Pandas、SciPy、scikit-learn	精通
数据存储	SQLite、SQL、Redis 基础	熟练
DevOps	Docker、Git、Linux、CNB 云原生	熟练
前端	React、TypeScript、D3.js	掌握

五、选择理由与贡献意愿

为什么选择 tRPC？

第一，技术匹配度高。我此前在 tRPC-Agent 框架上的前置学习不是“看过文档”的程度，而是逐行读过核心源码、动手改过代码、提交过功能开发的深度。这个课题的租户模型设计、存储抽象层、Filter 扩展，恰好是我已经接触过的领域的延伸。

第二，这个课题有真实的工程价值。“多租户”和“节点部署”不是一个学术问题，而是任何一个想要把 Agent 框架推向生产环境的项目都会遇到的现实问题。我不希望做完一个课题后产出只停留在仓库里，而是希望它真的能被用到实际场景中。

贡献意愿：

开源贡献不是一次性的 PR，而是持续的参与。我希望通过这个课题深入理解 tRPC-Agent 的服务化架构，成为社区活跃的贡献者。具体来说：

- 课题完成后持续跟进 PR 评审，将实现落地到框架主干
- 长期参与 tRPC 社区的 Issue 讨论和代码贡献
- 将实战经验沉淀为文档，帮助更多开发者上手

总结：我已在路上，不是从零开始。前置学习、源码阅读、动手实验、功能开发——每一步都留下了可验证的证据。这个课题是我现有能力的一个自然延伸，也是我期待在 tRPC 社区长期深耕的方向。

申请项目：tRPC（第一志愿） · Tencent YOLO-Master（第二志愿） · WeKnora（第三志愿）